ideally support generalization to related tasks. For any $t \in \mathcal{T}$, we assume access to a set of training instances (s, o) drawn from some distribution P_t (t itself may be drawn from $P_{\mathcal{T}}$). For example, a task to compare two movies might use token/domain-embedding input sequences as shown in Fig. 2, and output a textual comparison of the two movies represented by the input embeddings. In general, this does not require knowledge of the item v which corresponds to the input embedding vector $E_D(v)$, but in practice, one will use v to form target output training sequences.

Architecture. To account for the embedding space $\mathcal{W}$, we augment the pretrained $\mathcal{M} = (E_0^H, M_0)$ with an adapter model $E_A : \mathcal{W} \mapsto \mathcal{Z}$ to create a new language model $\mathcal{M}_{\text{ELM}} = ((E_0 \times E_A)^H, M_0)$. Here, E_0 is the usual embedding layer, which maps tokens representing textual inputs to $\mathcal{Z}$, whereas E_A maps embeddings from the latent metric space $(\mathcal{W}, d)$ to $\mathcal{Z}$. Importantly, E_0 and E_A map tokens and embeddings (respectively) to a common space[4] (see Fig. 2).

Training Procedure. Given a downstream loss function $\mathcal{L}$, we can differentially optimize the model $\mathcal{M}_{\text{ELM}}^{\theta}$ with parameters θ over the tasks in $\mathcal{T}$ by solving $\arg\min_{\theta} \mathbb{E}_{s,o \sim P_t; t \sim P_{\mathcal{T}}}[\mathcal{L}_{\theta}(\mathcal{M}_{\text{ELM}}(s, o))]$. Aside from maximum likelihood-based training, we discuss optimizing $\mathcal{M}_{\text{ELM}}$ using reinforcement learning from AI feedback (Lee et al., 2023) in Appendix H.

Training continuous prompts poses challenges due to constraints over the pretrained token embeddings, which may result in convergence to local minima (Liu et al., 2021). Intuitively, as the pretrained embedding layer E_0 readily maps to language space, the newly initialized embedding layer E_A may require numerous updates to map to a similar space. To overcome this, we divide training into two stages. In the first stage, we train the adapter E_A on tasks in $\mathcal{T}$ by keeping all other parameters (E_0, M_0) frozen. Since M_0 is pretrained, the learned first-stage mapping from $\mathcal{W}$ to $\mathcal{Z}$ improves convergence in the next stage. In the second stage, we fine-tune the full model by training all parameters (E_0, M_0, E_A). To increase efficiency, we can update a smaller number of parameters in alternating fashion (Hu et al., 2021). We found that two-stage training is essential for convergence of $\mathcal{M}_{\text{ELM}}$ (see Appendix D for further discussion of the two-stage training procedure).

3 Experiment Design

In this section, we describe the datasets, tasks, and evaluation methods used to train and validate ELM. We use the MovieLens 25M dataset (Harper & Konstan, 2015), which we enrich with textual descriptions by generating a large corpus of text using a PaLM 2-L (Unicorn) LLM (Google et al., 2023). We use two different forms of embeddings: *behavioral embeddings* and *semantic embeddings*, which we describe below. We also adopt various evaluation techniques for assessing the quality of ELM's outputs on test data, including qualitative human evaluations and specific consistency metrics.

3.1 Data

The MovieLens 25M dataset contains 25 million ratings (1 to 5) of 62,423 movies and 162,541 users.

Domain Embeddings. We create embeddings for both users and movies in the MovieLens dataset. We consider two types of embeddings. The first are *behavioral embeddings*, trained based solely on user ratings of movies. More generally such models can reflect any behavioral interaction between users and items, but use no direct semantic information. To train behavioral embeddings, we use *matrix factorization (MF)* computed using *weighted alternating least squares (WALS)* (Hu et al., 2008), such that the dot product $\hat{r} = \langle w_u^b, w_m^b \rangle$ of the embeddings of user u and movie m is predictive of user u's rating for movie m.[5] We train MF using movies that have at least five ratings.

Semantic embeddings are the second type, and are generated using textual descriptions of movies. Specifically, we use a pretrained dual-encoder language model (DLM) similar to *Sentence-T5* (Ni et al., 2022a) and *generalizable T5-based dense retrievers* (Ni et al., 2022b). More specifically, we concatenate plot descriptions and reviews for each movie, and input these to the DLM. We then average the resulting output vectors to generate the semantic embeddings. We denote movie m's semantic embedding by w_m^s.

[4]More expressive adapters that map vectors in $\mathcal{W}$ to length $\ell \geq 1$ sequences are left for future work.

[5]Other CF methods could be used, e.g., dual encoders (Yi et al., 2019; Yang et al., 2020), but our approach is agnostic to the precise method used to generate behavioral or semantic embeddings.

Training Data and Tasks. We test two versions of ELM, one that interprets semantic embeddings of movies, and a second which interprets behavioral embeddings of users. For the first model, we use a variety of tasks to test the model's ability to extract reasonable semantic information from the domain embeddings. More specifically, we construct 24 movie-focused tasks using a pretrained PaLM 2-L model. For each task, we generate training data by prompting the PaLM 2-L with a movie's title and additional task-specific information (e.g., writing review, listing characteristics, comparing movies, see details below). We then use PaLM 2-L's generated output as training targets for ELM. As for training inputs, we provide ELM with the same prompts used in PaLM 2-L, where the title of the movie m is replaced by its semantic embedding w^s_m. We emphasize that ELM does not receive any information about the movie (including its title), apart from its semantic embedding vector.

Our 24 movie tasks include single movie semantic tasks, such as describing a movie plot or summarizing a movie; single movie subjective tasks, such as writing positive or negative reviews for a movie; and movie pair subjective tasks, such as comparing characteristics of movies. Appendix E provides a complete description of all 24 tasks, including the prompts used, and sample outputs.

Finally, we train an additional model to interpret user behavioral embeddings, by generating *user preference profiles*. For this task, we generate a textual summary of a user's preferences as follows: (a) we sample five positively rated and five negatively rated movies for each user in MovieLens 25M; (b) we then prompt PaLM 2-L to describe (in ten bullet points) characteristics of a person who likes the first five movies, but dislikes the second five. This resulting summary is used as training output for ELM on the *user-profile generation task*. The resulting task is to generate such a user profile from a user's behavioral embedding w^b_u (i.e., interpret the domain embedding). Similar to the movie tasks, no other information about the user, apart from its embedding, is provided to ELM. We refer the reader to Appendix E for more details and examples.

3.2 Evaluation Metrics

We employ several forms of evaluation for ELM. First, given the inherent subjectivity of some tasks (e.g., movie reviews), human raters play a key role in gauging output quality. We ask raters to assess ELM's output w.r.t. its consistency with a movie's plot, its linguistic coherence, and overall task quality. Each output is rated on a scale from 0 (completely irrelevant/incoherent) to 1 (highly relevant/coherent). The average rater's score provides a holistic assessment of ELM's performance.

Second, since our goal is interpretation of embeddings spaces, and should allow the generation of descriptions of domain embedding vectors for which no entity exists in the domain vocabulary (e.g., hypothetical movies), we evaluate the *consistency* of ELM's generated text with the underlying domain embedding, as well as ground-truth data used to train the domain embedding. To achieve this, we introduce two consistency metrics; namely, semantic and behavioral consistency.

Semantic consistency (SC) compares the semantic embedding of generated text with the original embedding that produced that text. In our movie tasks, this metric is computed by *re-embedding* ELM's output into the same semantic space used to embed the original movies. Formally, we define the semantic consistency of an output o generated by an embedding $w^s \in \mathcal{W}$ by

$$\text{SC}(o, w^s) = d(P(o), w^s),$$

where $P(o)$ projects the output to the domain embedding space $\mathcal{W}$ and d is the distance metric in $\mathcal{W}$ (or alternatively, a similarity score); see Appendix B, Fig. 6 for an illustration.[6] The utility of SC is grounded in the expectation that if the generated output o holds true to its source w^s, then its embedded representation should not deviate substantially from its generating embedding vector. Hence, high SC suggests ELM has well-aligned domain embedding vectors with their (task-specific) descriptions. Importantly, this approach is not tied to examples in the data, providing the flexibility to measure the semantic consistency of *hypothetical entities*.

A second form of consistency, which we term *behavioral consistency* (BC), measures the ability to use ELM's output text to make *good behavioral predictions*. For example, in our user-profile task, we can test the ability of ELM's generated user profile to predict the movie preferences of a user corresponding to the input behavioral embedding. This in turn would suggest the extent to which the user profile captures the information implicit in that domain vector (i.e., the latent user preferences).

[6] In our experiments we use the same DLM to serve as P, i.e., to re-embed output text into semantic space.

Table 2: Test Scores for 24 movie tasks. We use cosine similarity as our metric for SC, and Spearman rank correlation coeff. for BC. For human evaluation, we show normalized scores between 0 (strongly disagree) and 1 (strongly agree). Table is divided into two sections: single movie tasks and two movie tasks. Tasks with nn abbreviation use nearest neighbor movies in semantic embedding space. A comparison of semantic and behavioral consistency metrics is demonstrated in Fig. 4.

Task (Appendix E)	Consistency Metrics		Human Evaluation (100 raters)		
	Semantic (Cosine)	Behavioral (Rank Corr.)	Consistency with Plot	Language Comprehensiveness	Task Quality
summary	0.91	0.94	0.87	0.87	0.85
positive review	0.94	0.81	0.9	0.9	0.89
negative review	0.93	0.79	0.93	0.96	0.96
neutral review	0.92	0.76	1	1	0.56
five pos char.	0.91	0.8	0.84	0.92	0.95
five neg char.	0.89	0.8	0.81	0.92	0.89
long description	0.91	0.8	0.89	0.86	0.89
funnier	0.88	0.81	0.75	0.78	0.68
sadder	0.91	0.82	0.59	0.83	0.81
scarier	0.9	0.84	0.69	0.74	0.95
improve	0.91	0.82	0.85	0.89	0.87
movie to viewer	0.89	0.8	0.89	0.83	0.86
pitch	0.96	0.8	0.96	0.95	0.94
criticize	0.89	0.77	0.88	0.7	0.79
convince1	0.92	0.96	0.98	0.93	0.77
convince2	0.9	0.88	0.89	0.86	0.87
convince3	0.88	0.86	0.7	0.67	0.74
dissuade1	0.88	0.85	0.88	0.89	0.87
dissuade2	0.88	0.87	0.79	0.8	0.79
similarities	0.86	0.83	0.37	0.4	0.38
interpolation	0.89	0.8	0.85	0.82	0.84
why like nn	0.83	0.48	0.84	0.84	0.91
diff than nn	0.82	0.48	0.61	0.79	0.64
common with nn	0.77	0.46	0.91	0.86	0.93

To achieve this, we assume access to some "off-the-shelf" language-based retriever/ranker ρ which, given a user profile, can rank a set of candidate movies. Then, given a fixed set of target movies, $\mathcal{I}$, and letting o_u be the ELM-generated user profile given domain embedding w_u^b, we define

$$\text{BC}(o_u, u) = \text{RankingScore}(R_u, \rho(o_u)),$$

where $\rho(o_u)$ are the ranks of the movies in $\mathcal{I}$ for user u provided by the language-based ranker, and $R_u = \{r(u, m) : m \in \mathcal{I}\}$ are the (MovieLens) ground-truth ratings for $\mathcal{I}$ provided by u (converted to rankings); see Appendix B, Fig. 7 for an illustration.[7] Here, *RankingScore* is any rank-comparison metric which compares $\rho(o_u)$ to the ground-truth ranking R_u. Examples include *NDCG* (Burges et al., 2005) and the *Spearman rank-correlation coefficient*. Finally, we measure behavioral consistency of movies in a fully analogous fashion (see Appendix B for a formal definition).

4 Empirical Analysis

We evaluate ELM on our 24 movie tasks and the user profile task, described in Sec. 3.1, using human raters, and semantic and behavioral consistency to measure performance. We analyze test results on both real entities (i.e., held-out movies) and hypothetical embedding vectors not in the data, to test ELM's ability to interpolate between movies or users, as well as extrapolate movie or user attributes.

We train two models: one (joint) model for the 24 movie tasks, and another for user profiles. For the movie tasks, we use 1000 randomly sampled examples for test and the rest for training. For the user profile task we use an 80/20 training/test split. Both models are fine-tuned using a pretrained

[7] In our experiments we use the dual-encoder DLM to serve as ρ, by embedding test movies and user profiles and using their similarity for scoring/ranking.